



TRACE: Traceable Real-time Adaptive Continual Explainable Deep Fake Detection System

DISUSUN OLEH:

Vissuta Gunawan Lim (Universitas Indonesia)

Andi Muhammad Adlyn Fakhreyza Khairi Putra (Universitas Indonesia)

Argya Farel Kasyara (Universitas Indonesia)

Latar Belakang

Perkembangan teknologi kecerdasan buatan telah membuka kemungkinan tak terbatas bagi pengguna untuk melakukan utilisasi teknologi yang dapat mempermudah kepentingan pribadi maupun kolektif. Kendati demikian, kemudahan ini juga hadir bersamaan dengan tantangan menghadapi manipulasi realitas yang mampu dilakukan teknologi kecerdasan buatan jika berada pada tangan yang salah. Salah satu bentuk manipulasi yang paling mengkhawatirkan adalah *deep fake*, teknologi yang memungkinkan pengguna untuk memanipulasi dan menciptakan foto atau video (audio-visual) secara sintetik yang cukup meyakinkan di mata manusia awam. [1] Hal ini dibuktikan oleh laporan Sumsu (2023), sebuah layanan verifikasi identitas terkemuka, yang mengungkapkan peningkatan drastis jumlah *deep fake* global sebesar sepuluh kali lipat dari tahun 2022 ke 2023, dengan peningkatan tertinggi terjadi di Amerika Utara (1.740%) dan Asia Pasifik (1.530%), namun dengan gencarnya peredaran konten *deep fake*, nyatanya sebanyak 48.2% responden dalam studi terkait belum bisa membedakan konten *deep fake* dengan konten asli (PNAS, 2024).

Kurangnya kemampuan masyarakat dalam membedakan konten asli dengan *deep fake* menimbulkan konsekuensi serius di berbagai bidang, mulai dari penyebaran disinformasi yang merusak kepercayaan publik, manipulasi sentimen pasar melalui replikasi tokoh kunci perusahaan, hingga destabilisasi demokratis melalui manipulasi figur publik. Dalam upaya menghadapi ancaman tersebut, para ahli telah mengembangkan berbagai solusi, di antaranya [2] dataset FaceForensics++ oleh Rossler et al. untuk pelatihan dan evaluasi model, [3] model berbasis CNN oleh Yasser et al. (2023), [4] metode ekstraksi fitur AVFF (*Audio-Visual Feature Fusion*) oleh Oorloff et al. (2024) untuk integrasi analisis audio-visual, dan [5] kerangka kerja evaluasi *explainability* oleh Tsigos et al. (2024).

Penelitian dan pengembangan sistem deteksi *deep fake* telah menunjukkan kemajuan signifikan, namun masih memiliki beberapa keterbatasan kritis. Sistem yang ada umumnya hanya melakukan inferensi secara *post-processing* yang membuka celah waktu dalam penyebaran konten manipulatif, tetapi tidak memiliki kemampuan pelacakan wajah asli, serta belum mengintegrasikan penjelasan komprehensif mengenai indikator-indikator yang mendasari hasil deteksi. Berdasarkan permasalahan tersebut, TRACE (**T**raceable **R**ead-time **A**daptive **C**ontinual **E**xplainable **D**eep **F**ake Detection System) dikembangkan sebagai sistem deteksi *deep fake* yang menerapkan deteksi *real-time* untuk mencegah penyebaran konten manipulatif, dilengkapi dengan kemampuan pembelajaran berkelanjutan untuk beradaptasi dengan teknik *deep fake* terbaru. Sistem ini juga mengimplementasikan fitur pelacakan wajah asli dan penjelasan terperinci yang tidak hanya mengklasifikasikan konten, tetapi juga menjelaskan alasan klasifikasi tersebut, sehingga membuka sifat *black box* pada sistem deteksi *deep fake*.

sekaligus memberikan wawasan kepada masyarakat tentang ciri-ciri konten manipulatif.

Tujuan dan Manfaat

Pengembangan sistem TRACE bertujuan untuk menciptakan solusi komprehensif dalam mengatasi ancaman *deep fake* melalui sistem deteksi yang dapat diandalkan, adaptif, dan transparan. Secara spesifik, sistem ini dikembangkan untuk mencapai empat tujuan utama: menghadirkan kemampuan deteksi *deep fake* secara *real-time* untuk mencegah penyebaran konten manipulatif sedini mungkin, membangun sistem pembelajaran berkelanjutan yang dapat beradaptasi dengan teknik-teknik manipulasi terbaru tanpa mengorbankan kemampuan deteksi sebelumnya, mengintegrasikan kemampuan pelacakan wajah asli untuk mendukung proses investigasi konten *deep fake*, serta mengimplementasikan mekanisme penjelasan yang komprehensif untuk meningkatkan kepercayaan pengguna terhadap hasil deteksi.

Implementasi sistem TRACE diharapkan dapat memberikan manfaat signifikan bagi berbagai pemangku kepentingan dalam upaya memerangi penyalahgunaan teknologi *deep fake*. Bagi masyarakat umum, sistem ini menyediakan alat verifikasi yang dapat diandalkan untuk memvalidasi keaslian konten digital secara instan, sehingga mengurangi risiko terpapar dan terpengaruh oleh konten manipulatif. Bagi platform media sosial dan penyedia layanan konten digital, fitur deteksi *real-time* dan pembelajaran berkelanjutan memungkinkan penyaringan konten yang lebih efektif dan adaptif. Bagi penegak hukum dan jurnalis, kemampuan pelacakan wajah asli dan penjelasan terperinci melalui modul *explainability* berkontribusi pada proses investigasi dan verifikasi yang lebih akurat. Secara lebih luas, transparansi sistem dalam menjelaskan hasil deteksi berpotensi meningkatkan literasi digital masyarakat terkait manipulasi konten sehingga mendorong terbentuknya ekosistem informasi digital yang lebih sehat dan terpercaya.

Solusi

TRACE (*Traceable Real-time Adaptive Continual Explainable Deep Fake Detection System*) dirancang sebagai solusi komprehensif untuk melakukan deteksi konten *deep fake*, baik berupa gambar, video prarekam, maupun konten *real-time*. TRACE mempunyai dua alur pemrosesan yang berbeda untuk menangani *input* gambar dan audio-visual (video prarekam dan *real-time*). Dua alur pemrosesan tersebut mempunyai dua *database* yang berbeda pula, yaitu *Image Database* untuk menyimpan data berupa gambar dan *Video Database* untuk menyimpan data video. (Lihat Figur 1).

Pada *initial state*, *Image Database* diisi dengan [1] dataset *open source* FaceForensics++ yang terdiri dari lebih dari 500.000 gambar yang merupakan *frame* hasil ekstraksi dari 1000 video asli dan hasil manipulasi menggunakan empat

teknik manipulasi yang berbeda, yaitu Face2Face, Deepfakes, FaceSwap, dan NeuralTextures. Sementara itu, *Video Database* diisi dengan [6] dataset *open source* FakeAVCeleb yang terdiri dari 500 video subjek asli dan lebih dari 20.000 video hasil manipulasi yang dihasilkan oleh empat *framework deep fake* yang berbeda: FakeYou, FakeYouAudio, Wav2Lip, dan LipGAN.

Di sisi lain, untuk mencapai *initial state*, kedua *encoder* melewati tahap pelatihan yang berbeda. *Image Encoder* menggunakan arsitektur berbasis Convolutional Neural Network (CNN) yang terdiri dari lima blok konvolusional, dimana setiap blok konvolusional terdiri dari beberapa komponen: lapisan konvolusi $N \times N$, *Instance Normalization* untuk menormalkan aktivasi setiap fitur secara independen, dan fungsi aktivasi ReLU (*Rectified Linear Unit*). Setiap blok konvolusional diikuti oleh operasi *max pooling* 2×2 untuk mereduksi dimensi spasial secara progresif. Pada tahapan terakhir, hasil ekstraksi fitur diratakan (*flatten*) dan diproses melalui lapisan *fully connected* untuk menghasilkan vektor fitur berdimensi 512 yang merepresentasikan karakteristik visual dari gambar input. Pada tahap pelatihan, *Image Encoder* dilatih menggunakan dataset FaceForensics++ dengan tujuan untuk membedakan gambar asli dan gambar hasil manipulasi. Proses pelatihan (Algoritma 1 pada Lampiran) menggunakan fungsi *binary cross-entropy loss* untuk mengoptimalkan kemampuan ekstraksi fitur yang diskriminatif antara konten asli dan *deep fake*. Dataset dibagi menjadi data latih dan data validasi dengan rasio 80:20, dimana proses pelatihan dilakukan secara iteratif menggunakan metode *mini-batch gradient descent* dengan ukuran *batch* 32 dan *learning rate* awal 0.001. Proses pelatihan berlangsung selama 30 epoch dengan menggunakan *ReduceLROnPlateau scheduler* yang menurunkan *learning rate* dengan faktor 0.1 ketika performa model stagnan selama 5 epoch.

Tahap pelatihan *Audio-Visual Encoder* terdiri dari dua fase. Fase pertama adalah pembelajaran representasi mandiri (*self-supervised representation learning*) (Figur 6) menggunakan [7] dataset LRS3, dimana model belajar memahami korespondensi natural antara komponen audio dan visual. Proses ini menggunakan pendekatan *masked token modeling* dengan membagi input menjadi K potongan temporal, dimana 50% potongan dimasking secara acak. Model dilatih menggunakan fungsi *loss* ganda: *contrastive loss* untuk mengukur kesesuaian antara representasi fitur audio-visual dan *autoencoding loss* untuk akurasi rekonstruksi token yang di-*masking*. Setelah fase ini selesai, dilanjutkan dengan fase kedua yaitu *supervised learning* (Algoritma 2 pada Lampiran) menggunakan dataset FakeAVCeleb untuk tugas deteksi *deep fake* audio-visual. Sama seperti *Image Encoder*, fase ini menggunakan *binary cross-entropy loss* dengan konfigurasi pelatihan serupa namun dengan 20 epoch dan menggunakan *CosineAnnealingLR scheduler* yang menurunkan *learning rate* secara bertahap mengikuti fungsi cosinus.

Pelatihan juga dilakukan pada *Disentanglement Reversing Network* (DRN), yang mekanismenya akan dijelaskan lebih detail pada paragraf mendatang, untuk mencapai *initial state* menggunakan dataset FaceForensics++. Pada tahap pelatihan, jaringan dilatih menggunakan input gambar berukuran 128×128 piksel

dengan fungsi loss total (L) yang merupakan kombinasi terbobot dari enam komponen *loss*: L_{id} untuk ekstraksi fitur identitas, L_{redun} untuk meminimalisasi redundansi, L_{recon} untuk rekonstruksi piksel, L_{map} untuk pemetaan identitas, L_{gen} untuk generasi gambar, dan L_{cycle} untuk konsistensi siklus. Bobot λ_1 hingga λ_5 diatur ke nilai 1, sementara λ_6 diberi bobot lebih tinggi yaitu 5 untuk memprioritaskan preservasi identitas.

Proses pelatihan dilaksanakan menggunakan *optimizer* Adam dengan learning rate 0.0003 dan momentum (β_1 , β_2) sebesar 0.5 dan 0.999. Model dilatih selama 200 iterasi dengan batch size 32, menggunakan LeakyReLU sebagai fungsi aktivasi non-linear dan metode Kaiming untuk inisialisasi bobot. Arsitektur ini mengadopsi struktur UNet yang dimodifikasi dengan penambahan layer *flatten* dan *fully connected* setelah proses *down-sampling* untuk menghasilkan vektor fitur yang dapat digunakan dalam supervisi identitas dan redundansi. Setelah melatih *Image Encoder*, *Audio Visual Encoder*, dan *Disentangling Reversing Network*, *initial state* sudah dicapai.

Dari tahap *initial state* tersebut, TRACE siap digunakan untuk mendeteksi konten *deep fake*. Pengguna dapat memberikan input berupa gambar, video, atau konten *real-time* yang akan diproses melalui dua *pipeline* yang berbeda. Input berupa gambar akan diproses melalui *Image Deep Fake Detection Pipeline*, sedangkan input berupa video prarekam dan *live screen capture* akan diproses melalui *Audio-Visual Feature Fusion Pipeline*.

Image Deep Fake Detection Pipeline (Lihat Figur 2) menggunakan *Image Encoder* terlatih sebagai modul ekstraksi fitur dengan proses *forward propagation* yang sistematis. Input gambar dengan dimensi 3×128×128 (RGB) diproses melalui lima blok konvolusional terlatih, di mana setiap blok menerapkan konvolusi N×N, *Instance Normalization*, dan aktivasi ReLU, diikuti dengan *max pooling* 2×2 yang secara progresif mereduksi dimensi spasial dengan stride 2. Hasil ekstraksi fitur diratakan menjadi vektor berdimensi 512 yang merepresentasikan karakteristik visual mendalam dari input. Vektor ini kemudian diproses melalui jaringan klasifikasi yang terdiri dari transformasi linear FC(512→256) dengan aktivasi ReLU, regularisasi *dropout* dengan probabilitas 0.5 untuk mencegah *overfitting*, dan transformasi linear terakhir FC(256→2) dengan aktivasi *softmax* yang menghasilkan distribusi probabilitas $p(y | x) \in [0, 1]^2$ untuk klasifikasi biner asli (0) atau palsu (1).

Audio Visual Feature Fusion Pipeline (Lihat Figur 3) mengimplementasikan arsitektur fusi multi-modal untuk deteksi *deep fake* berbasis ketidaksesuaian audio-visual. Video input pertama di-demultiplex menjadi komponen audio (x_a) dan visual (x_v). Komponen audio diproses oleh *Audio Encoder* (E_a) menghasilkan embedding $a \in \mathbb{R}^d$, sementara frame-frame visual diproses oleh *Visual Encoder* (E_v) menghasilkan embedding $v \in \mathbb{R}^d$, dengan d adalah dimensi ruang fitur. Kedua embedding ini kemudian diproses melalui jaringan lintas modalitas: A2V menghasilkan proyeksi audio-ke-visual $v_a = A2V(a)$, dan V2A menghasilkan proyeksi

visual-ke-audio $a_v = V2A(v)$. Representasi dari masing-masing modalitas digabungkan melalui operasi konkatenasi sepanjang dimensi fitur, menghasilkan $f_v = [v; a_v]$ dan $f_a = [a; v_a]$. Fitur tergabung ini diproses melalui jaringan *patch reduction* (ψ_a, ψ_v) untuk mereduksi dimensi temporal sebelum diklasifikasikan oleh *classifier head* Γ yang menghasilkan logit $l = \Gamma(\psi_a(f_a) \oplus \psi_v(f_v))$.

Untuk pemrosesan *real-time*, pipeline dimodifikasi untuk menangani stream video secara kontinyu. Video stream dibagi menjadi blok temporal dengan durasi T dan overlap $T/8$ untuk memastikan kontinuitas deteksi. Setiap blok diproses secara independen dengan output logit dirata-ratakan secara temporal untuk menghasilkan deteksi yang stabil. Berbeda dengan video prarekam yang dapat diproses secara batch, pemrosesan real-time membutuhkan optimasi khusus seperti *buffer circular* untuk manajemen memori dan teknik *pipeline parallelization* untuk memenuhi konstrain latensi. Sistem menjaga *buffer frame* berukuran T untuk memastikan konteks temporal yang cukup dalam analisis sambil mempertahankan latensi deteksi di bawah 100ms.

Hasil dari kedua *pipeline* kemudian menghasilkan *classifier* yang menghasilkan label biner (0/1) dan tingkat kepercayaan (*confidence score*) untuk setiap prediksi hasil dari fungsi *softmax*. Label dan input yang bersesuaian kemudian disimpan ke dalam *database* yang bersesuaian (*Image Database/Video Database*) untuk mendukung pembelajaran berkelanjutan. Namun, pembelajaran berkelanjutan (*continual learning*) konvensional seringkali menghadapi masalah *catastrophic forgetting*, di mana model kehilangan pengetahuan yang telah dipelajari sebelumnya ketika mempelajari pola baru. Untuk mengatasi hal ini, TRACE mengimplementasikan mekanisme *knowledge distillation*.

Dalam paradigma [9] *knowledge distillation*, model yang telah dilatih sebelumnya bertindak sebagai *teacher* yang mendistilasi pengetahuannya ke model baru (*student*). Proses ini melibatkan tiga komponen utama: pengetahuan dari model *teacher*, data baru yang akan dipelajari, dan mekanisme transfer pengetahuan (Figur 2). Model *teacher* yang telah dilatih dengan N *instance* menghasilkan distribusi probabilitas lunak (*soft probability distribution*) melalui fungsi *softmax* dengan temperatur T yang lebih tinggi:

$$p_i = \exp(z_i/T) / \sum_j \exp(z_j/T)$$

di mana p_i adalah probabilitas untuk kelas ke- i , z_i adalah logit output untuk kelas ke- i , $\sum_j$ adalah penjumlahan untuk semua kelas j dan T adalah parameter temperatur yang mengontrol 'kelembutan' distribusi. Model *student* kemudian dilatih untuk meminimalkan kombinasi dari dua fungsi *loss*:

$$L_{total} = \alpha \cdot L_{CE}(y, \sigma(z_s)) + (1 - \alpha) \cdot L_{KL}(\sigma_T(z_t), \sigma_T(z_s))$$

di mana L_{total} adalah total *loss* yang akan diminimalkan, L_{CE} adalah *cross-entropy loss* antara prediksi *student* dan label sebenarnya, L_{KL} adalah *Kullback-Leibler*

divergence antara distribusi lunak *teacher* dan *student*, y adalah label ground truth, z_s adalah logit output dari model *student*, z_t adalah logit output dari model *teacher*, σ adalah fungsi *softmax* standar dengan $T = 1$, σ_T adalah *softmax* dengan temperatur T , dan α adalah parameter yang menyeimbangkan kedua komponen *loss*. Pendekatan ini memungkinkan model baru untuk mempelajari pola dari *instance* baru ($N+1$) sambil mempertahankan pengetahuan yang telah didistilasi dari model sebelumnya tentang N *instance* pertama.

Di saat yang bersamaan, untuk meningkatkan transparansi dan kepercayaan terhadap hasil deteksi, TRACE mengimplementasikan dua mekanisme analisis lanjutan, antara lain:

[5] LIME (*Local Interpretable Model-agnostic Explanations*) digunakan untuk memberikan penjelasan visual menggunakan prinsip aproksimasi lokal, di mana model yang kompleks diaproksimasi oleh model yang lebih sederhana dan interpretable di sekitar prediksi tertentu. Secara teknis, LIME beroperasi dengan membagi gambar input menjadi segmen-segmen (*superpixels*) yang kemudian diperturbasi dengan menonaktifkan segmen secara acak untuk menghasilkan sampel-sampel perturbasi. Setiap perturbasi diberikan bobot w_i berdasarkan jarak kernelnya dengan input asli: $w_i = K(d(x, x'_i))$, di mana K adalah fungsi kernel dan d adalah fungsi jarak. Model asli $f(x)$ kemudian membuat prediksi untuk setiap sampel perturbasi x'_i , dan LIME melatih model linear $g(x)$ yang meminimalkan fungsi objektif:

$$L(f, g, w) = \sum_i w_i (f(x'_i) - g(x'_i))^2 + \Omega(g)$$

di mana $\Omega(g)$ adalah term regularisasi untuk menjaga kompleksitas model interpretable.

Adapun alasan dibalik penggunaan LIME untuk mendemonstrasikan *explainability* adalah karena signifikansinya dalam memvisualisasikan bagian wajah yang terindikasi *deep fake*. [5] Studi Tsigos et al. (2024) mengkonfirmasi efektivitas superior LIME dalam konteks deteksi *deep fake* dengan menunjukkan penurunan akurasi deteksi terbesar (hingga 0.245 untuk top-3 region pada kasus Deepfakes) ketika region yang diidentifikasi dimodifikasi, serta skor sufisiensi yang tinggi (mencapai 0.565 untuk Deepfakes) yang memvalidasi bahwa region yang diidentifikasi benar-benar merupakan indikator kunci dalam pengambilan keputusan model.

Selain LIME, TRACE juga mengadopsi arsitektur [8] Disentanglement Reversing Network (DRN) (Figur 4) yang terdiri dari dua modul utama: modul *identity disentangling* dan modul *face reversing* (Figur 5). Modul *identity disentangling* menggunakan encoder ganda (E_{id_ori} dan E_{attr}) untuk memisahkan fitur identitas dan atribut dari gambar input, yang kemudian digabungkan melalui decoder D untuk merekonstruksi wajah asli. Secara teknis, encoder E_{id_ori} menghasilkan vektor fitur f_{ori_id} berukuran 1×512 yang merepresentasikan identitas

wajah, sementara E_{attr} mengekstrak fitur atribut f_{ori_attr} yang kemudian dikonkatenasi menjadi f_{ori_conc} untuk rekonstruksi.

Pada modul *face reversing*, jaringan menggunakan pendekatan yang serupa namun dengan tujuan untuk menghasilkan I_{tra} dengan identitas yang sama seperti I_{ori} namun dengan atribut dari I_{fake} . Proses ini melibatkan ekstraksi identitas dan atribut dari I_{fake} menggunakan encoder yang sama, menghasilkan $f_{gen_ori_id}$ dan f_{fake_attr} yang kemudian digabungkan menjadi f_{fake_conc} . Untuk memastikan konsistensi siklus dan preservasi identitas, DRN menerapkan mekanisme supervisi ganda dengan menghitung loss antara fitur identitas yang diekstrak dari I_{tra} (tra_ori_id) dengan identitas asli.

Arsitektur ini diimplementasikan menggunakan struktur UNet yang dimodifikasi, dengan empat layer encoder dan lima layer decoder, serta mekanisme skip-connection untuk mempertahankan detail fitur pada berbagai skala. Input berukuran $3 \times 128 \times 128$ diproses melalui serangkaian conv block 3×3 dengan normalisasi instans dan aktivasi ReLU, menghasilkan feature map $1 \times 512 \times 16 \times 16$ yang kemudian dikonversi menjadi vektor fitur melalui layer flatten dan fully connected.

Untuk input video, frame yang memiliki tingkat ketidaksesuaian audio-visual tertinggi berdasarkan skor output dari jaringan A2V dan V2A dalam AVFF Pipeline akan dipilih untuk input DRN. Frame terpilih dihitung menggunakan jarak cosinus antara proyeksi audio-ke-visual (v_a) dan fitur visual asli (v), serta antara proyeksi visual-ke-audio (a_v) dan fitur audio asli (a), dimana frame dengan total jarak terbesar mengindikasikan ketidaksesuaian tertinggi antara modalitas audio dan visual.

Untuk mengoptimalkan performa dan skalabilitas sistem, TRACE diintegrasikan dengan platform Bliv dan Vertex AI. Integrasi dengan Bliv memungkinkan pengelolaan data yang lebih efisien, sementara Vertex AI menyediakan infrastruktur cloud yang handal untuk deployment model.

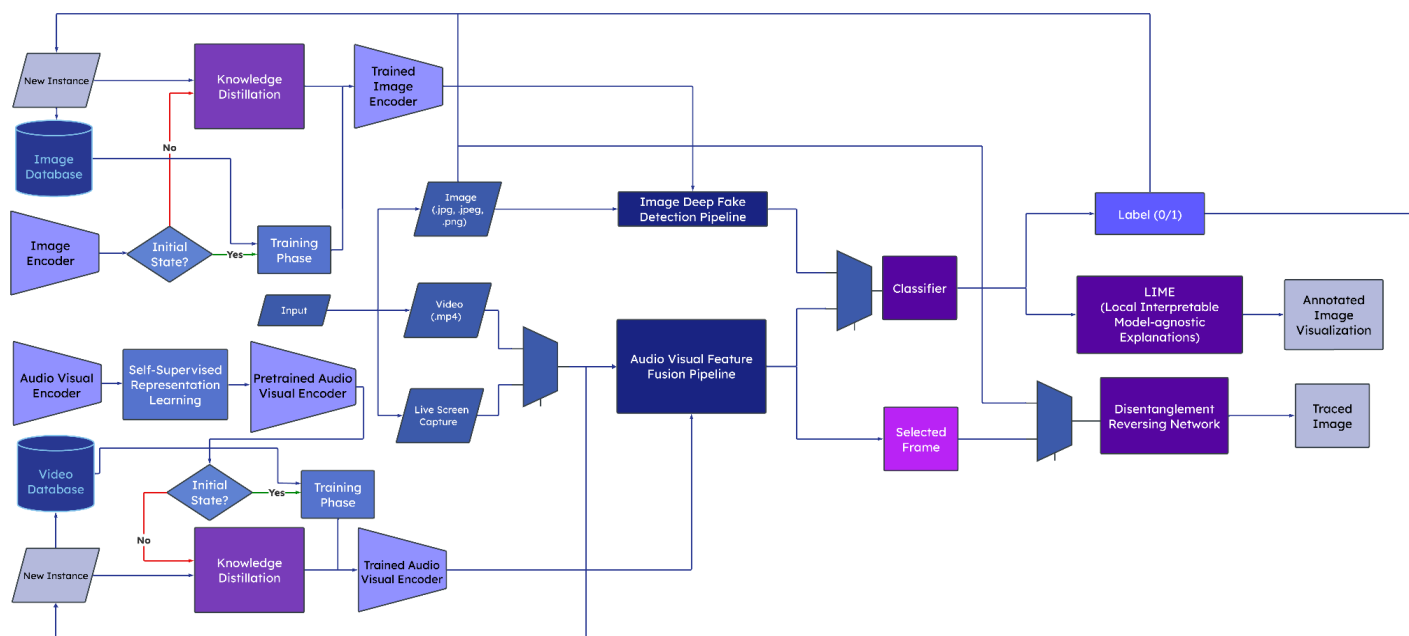
Integrasi dengan Bliv difokuskan pada tiga aspek utama: (1) manajemen data menggunakan Bliv ETL Pipeline untuk mengintegrasikan dataset dari berbagai sumber dan memastikan konsistensi format, (2) visualisasi real-time menggunakan Bliv Dashboard untuk monitoring performa sistem dan tren deteksi deep fake, dan (3) pemanfaatan Bliv AI untuk optimasi model berkelanjutan. Bliv ETL Pipeline secara khusus dikonfigurasi untuk menangani aliran data multimedia dengan throughput tinggi, menggunakan teknik data partitioning untuk memisahkan data gambar dan video, serta mengimplementasikan caching strategis untuk mengoptimalkan akses data frequently used.

Sementara itu, integrasi dengan Vertex AI memungkinkan TRACE untuk: (1) melakukan distributed training menggunakan infrastruktur cloud yang scalable, (2) mengoptimalkan hyperparameter secara otomatis menggunakan Vertex AI AutoML, dan (3) mengelola deployment model melalui Vertex AI Model Registry. Proses training model dilakukan menggunakan Vertex AI Training dengan

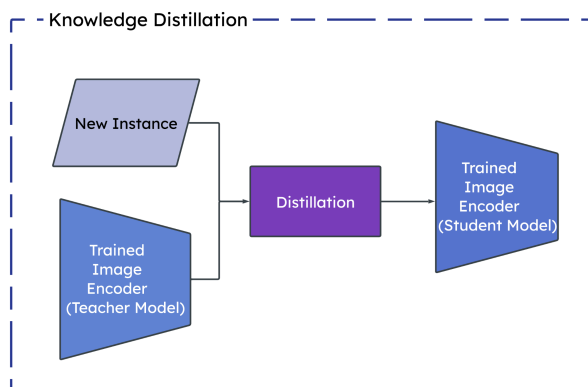
konfigurasi custom container yang mengenkapsulasi dependensi spesifik TRACE. Model yang terlatih kemudian di *deploy* ke Vertex AI Endpoint yang dapat diskalakan secara otomatis berdasarkan beban permintaan, dengan target latency maksimum 100ms untuk pemrosesan real-time.

Kedua platform ini bekerja secara sinergis dimana Bliv bertanggung jawab atas orkestrasi data dan monitoring sistem, sementara Vertex AI menangani aspek komputasi dan deployment model. Dengan arsitektur terintegrasi ini, TRACE dapat memproses ribuan permintaan secara konkuren sambil mempertahankan performa dan reliabilitas sistem. Selain itu, integrasi ini juga memfasilitasi implementasi strategi continual learning yang lebih efektif dengan memanfaatkan kapabilitas distributed computing dari kedua platform.

Arsitektur

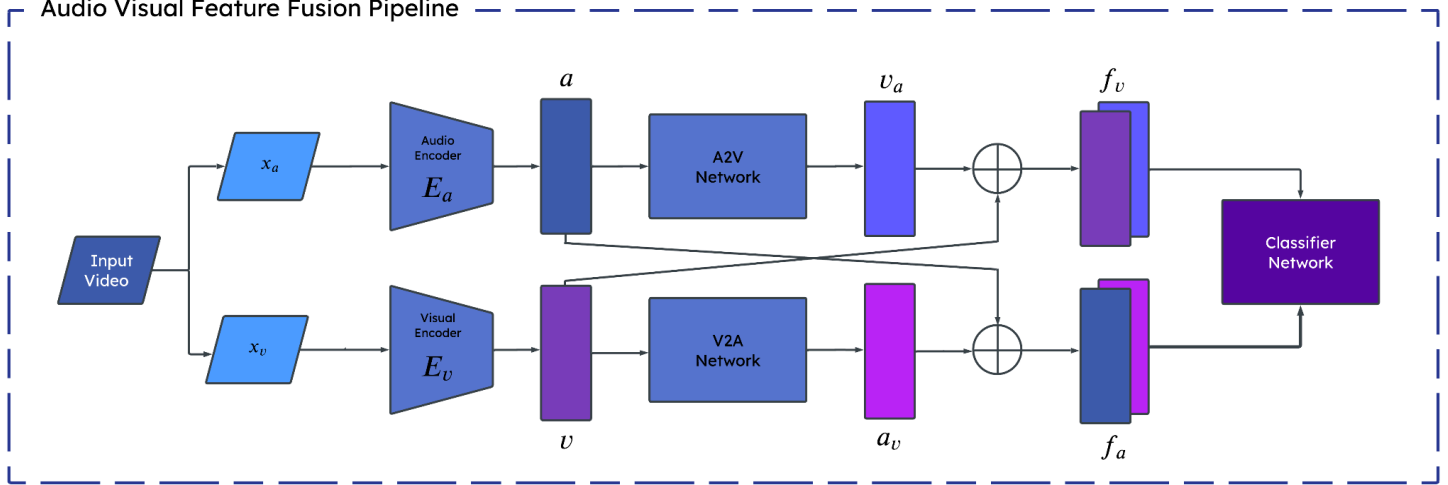


Figur 1. Diagram Utama TRACE



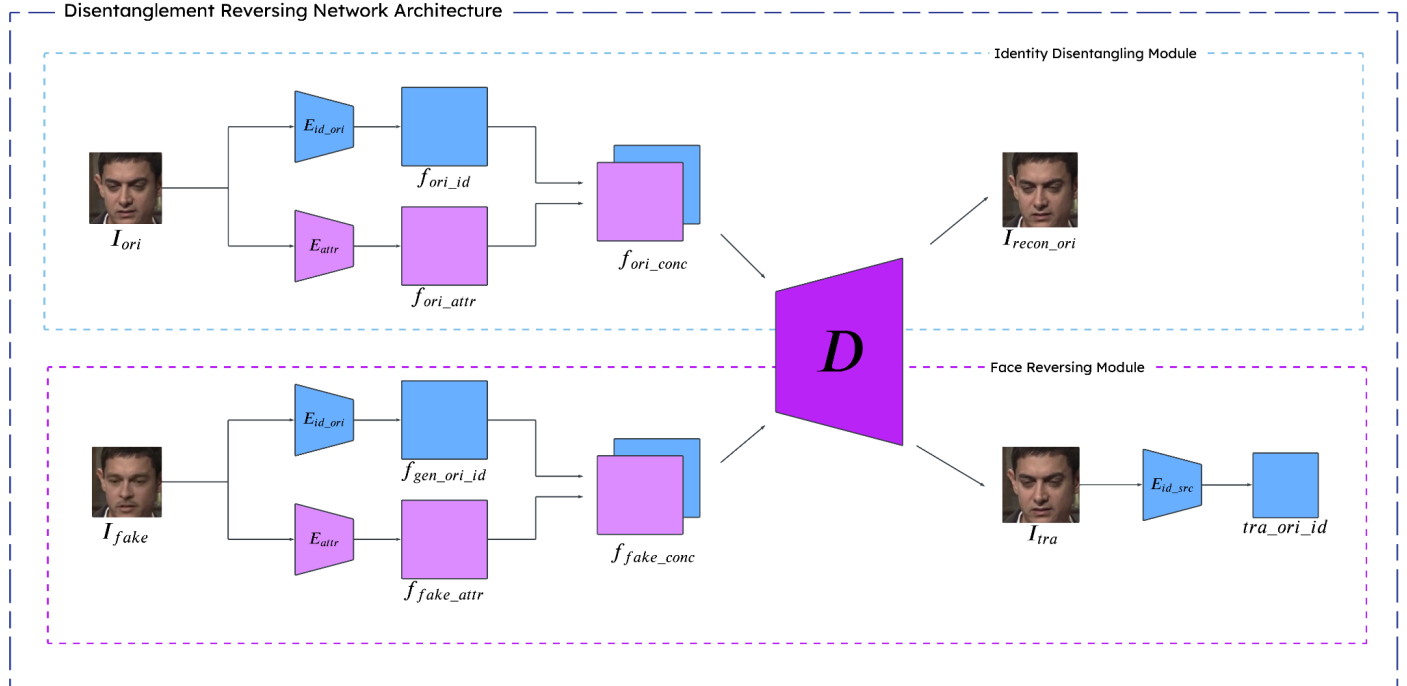
Figur 2. Ilustrasi Sederhana Knowledge Distillation

Audio Visual Feature Fusion Pipeline

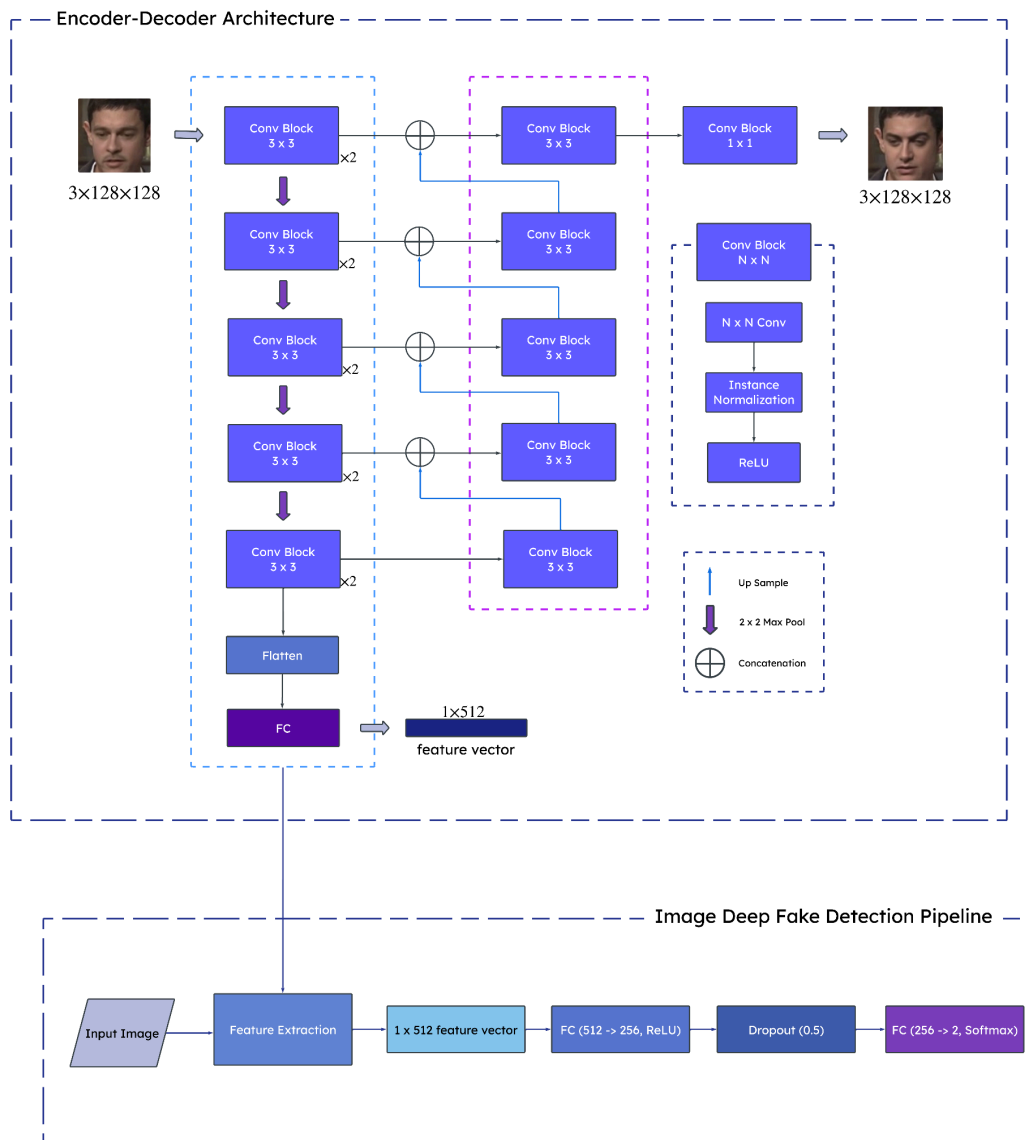


Figur 3. AVFF Pipeline

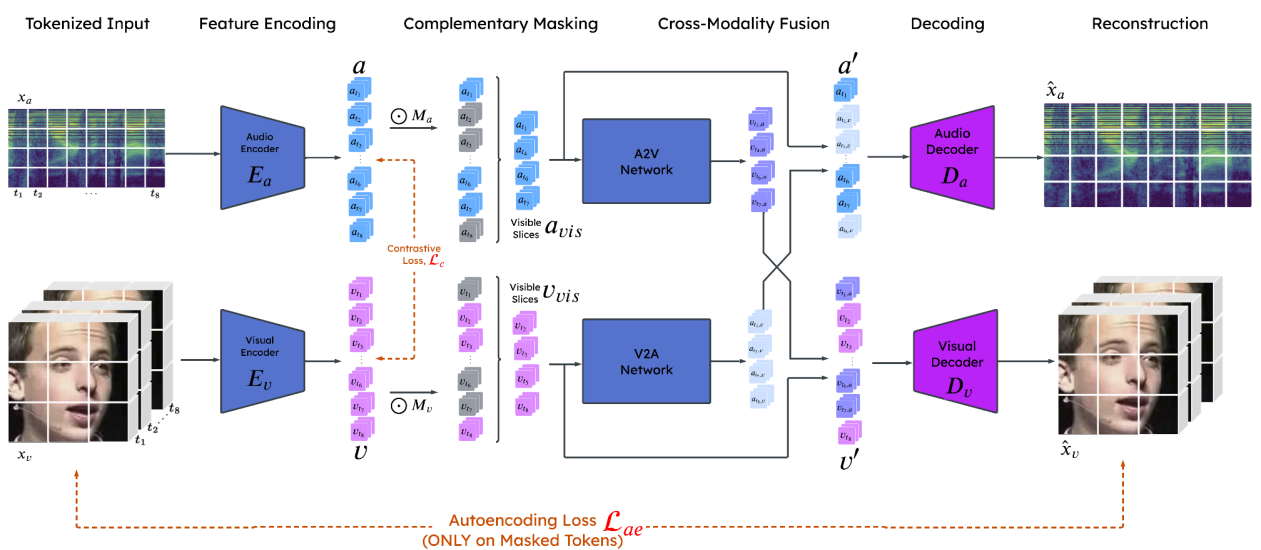
Disentanglement Reversing Network Architecture



Figur 4. Disentanglement Reversing Network



Figur 5. Encoder-Architecture untuk DRN dan Image Deep Fake Detection Pipeline



Figur 6. Self-Supervised Representation Learning Schema

Lampiran

Algoritma 1: Training Phase Image Encoder

Input: Dataset $D = \{(x_i, y_i)\}_{i=1}^N$, dimana x_i adalah gambar input dan y_i adalah label (0: asli, 1: deep fake)
Output: Model Image Encoder terlatih

1. Inisialisasi parameter model θ secara acak
2. Inisialisasi optimizer Adam dengan $\alpha = 0.001$
3. Inisialisasi ReduceLROnPlateau scheduler dengan patience=5, factor=0.1
4. Split dataset D menjadi D_{train} (80%) dan D_{val} (20%)
5. for epoch = 1 to 30 do
6. for setiap mini-batch $B \subseteq D_{\text{train}}$ do
7. Forward pass: prediksi $\hat{y} = \text{Encoder}(x; \theta)$
8. Hitung binary cross-entropy loss: $L = -\sum [y \log(\hat{y}) + (1-y)\log(1-\hat{y})]$
9. Backward pass: hitung gradien ∇L
10. Update parameter: $\theta = \text{optimizer.step}(\nabla L)$
11. end
12. $\text{val_loss} = \text{Evaluasi loss pada } D_{\text{val}}$
13. $\text{scheduler.step}(\text{val_loss})$ // Adjust learning rate
14. if akurasi meningkat then
15. Simpan model checkpoint
16. end
17. end
18. return Model dengan akurasi validasi terbaik

Algoritma 2: Training Audio Visual Encoder for Deep Fake Detection

Input: Dataset $D = \{(x_i, y_i)\}_{i=1}^N$, dimana x_i adalah video input dan y_i adalah label (0: asli, 1: deep fake)
Output: Model Audio Visual Encoder terlatih

1. Load pretrained encoder dari self-supervised learning
2. Inisialisasi optimizer Adam dengan $\alpha = 0.001$
3. Inisialisasi ReduceLROnPlateau scheduler dengan patience=5, factor=0.1

```
4. Split dataset D menjadi D_train (80%) dan D_val (20%)
5. for epoch = 1 to 20 do
6.     for setiap mini-batch  $B \subseteq D_{\text{train}}$  do
7.         Ekstrak fitur audio-visual dari video
8.         Forward pass: prediksi  $\hat{y} = \text{Encoder}(\text{audio}, \text{visual}; \theta)$ 
9.         Hitung binary cross-entropy loss:  $L = -\sum[y \log(\hat{y}) + (1-y)\log(1-\hat{y})]$ 
10.        Backward pass: hitung gradien  $\nabla L$ 
11.        Update parameter:  $\theta = \text{optimizer.step}(\nabla L)$ 
12.    end
13.    val_loss = Evaluasi loss pada D_val
14.    scheduler.step(val_loss) // Adjust learning rate
15.    if akurasi meningkat then
16.        Simpan model checkpoint
17.    end
18. end
19. return Model dengan akurasi validasi terbaik
```

Referensi

- [1] D. Ramirez, "70 Deepfake Statistics You Need To Know (2024)," Spiralytics, Sep. 26, 2024. [Online]. Available: <https://www.spiralytics.com/blog/deepfake-statistics/>
- [2] A. Rössler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Nießner, "FaceForensics++: Learning to Detect Manipulated Facial Images," in IEEE International Conference on Computer Vision (ICCV), Aug. 2019, pp. 1-10. [Online]. Available: <https://arxiv.org/pdf/1901.08971>
- [3] B. Yasser et al., "Deepfake Detection Using EfficientNet and XceptionNet," IEEE Trans. Pattern Anal. Mach. Intell., vol. 45, no. 11, pp. 13482-13497, Nov. 2023, doi: 10.1109/icicis58388.2023.10391114.
- [4] T. Oorloff et al., "AVFF: Audio-Visual Feature Fusion for Video Deepfake Detection," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), Jun. 2024, pp. 27092-27102, doi: 10.1109/cvpr5733.2024.02559.
- [5] K. Tsigos, E. Apostolidis, S. Baxevanakis, S. Papadopoulos, and V. Mezaris, "Towards Quantitative Evaluation of Explainable AI Methods for Deepfake

Detection," IEEE Trans. Artif. Intell., vol. 1, no. 2, pp. 156-169, Jun. 2024, doi: 10.1145/3643491.3660292.

[6] H. Khalid, S. Tariq, M. Kim, and S. Woo, "FakeAVCeleb: a Novel Audio-Video Multimodal Deepfake Dataset," arXiv preprint arXiv:2108.05080, Aug. 2021.

[7] T. Afouras, J. S. Chung, and A. Zisserman, "LRS3-TED: a Large-scale Dataset for Visual Speech Recognition," arXiv preprint arXiv:1809.00496, Oct. 2018.

[8] J. Ai et al., "Deepfake Face Traceability with Disentangling Reversing Network," arXiv preprint arXiv:2207.03666, Jul. 2022.

[9] G. Habib et al., "A Comprehensive Review of Knowledge Distillation in Computer Vision," arXiv preprint arXiv:2404.00936, Apr. 2024.